La Octava

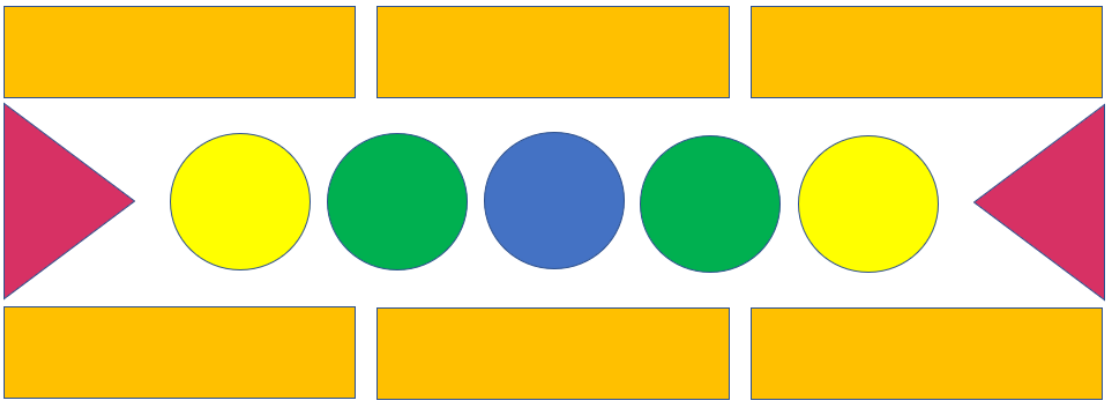
Autor: Luis Germán Gutiérrez Torres

Puntos	100	**Límite de memoria**	32 MiB
Límite de tiempo (caso)	1s	**Límite de tiempo (total)**	1m0s
Tamaño límite de entrada (bytes)	10 KiB		

Descripción

En Uriangato, Gto. existe una tradición desde hace muchos años llamada La Octava. Crean tapetes de aserrín como ofrenda a San Miguel. Estos tapetes pueden ser de diversas formas. Suponga que cada uno de los tapetes se compone solamente de algunas figuras geométricas: círculos, cuadrados, rectángulos y triángulos.

Ayuda a determinar el área total que cubren las figuras que forman un tapete en base a las figuras y medidas que se te proporcionarán. Por ejemplo, el tapete de la siguiente figura incluye 6 rectángulos, dos triángulos y 5 círculos. El área total que cubren las figuras es simplemente la sumatoria de todas sus áreas.



Tapete

El tipo de figura y los datos de entrada para cada una son los siguientes: 1.- Cuadrado, requiere la medida de un lado. 2.- Rectángulo, requiere lado1 y lado2. 3.- Círculo, requiere solo el radio. 4.- Triángulo, requiere base y altura.

Entrada

Recibirá un entero ***N*** que indica la cantidad de figuras del tapete, seguido ***N*** líneas donde cada una de las líneas contiene un entero ***T*** que indica el tipo de figura seguido por las dimensiones de la figura.

Salida

La suma total de las áreas de todas las figuras. Debe redondear el resultado final a 2 decimales.

Ejemplo

input	output	description
13 2 1 4 2 1 4 2 1 4 2 1 4 2 1 4 2 1 4 4 2 1.8 4 2 1.8 3 1.3 3 1.3 3 1.3 3 1.3 3 1.3	54.15	Explicación

input	output	description
2 1 2 2 5 4	24.00	Tipo 1: cuadrado, 2x2=4 Tipo 2: rectángulo, 5*4 =20 Total = 24.00

Límites

- ***1 <= N <= 100***
- ***1 <= T <= 4***
- Cada una de las dimensiones es un valor positivo menor a 1000.

Tetris

Autor: Ulises Méndez Martínez

Puntos	100	**Límite de memoria**	32 MiB
Límite de tiempo (caso)	1s	**Límite de tiempo (total)**	1m0s
Tamaño límite de entrada (bytes)	10 KiB		

Descripción

Antonio acaba de inventar un juego que nombró **Tetris**. Tetris se juega en una cuadrícula de tamaño $R \times C$. El objetivo del juego es llenar la cuadrícula para formar una configuración dada usando el **mínimo** número de piezas. Cada pieza es una **barra horizontal**, es decir de altura 1 y un ancho variable.

Ejemplo: Si se juega en una cuadrícula de 3×3 y la configuración objetivo es 1,2,1. Antonio solo necesita **2** piezas para alcanzar dicha configuración, la primera es una barra de 1×3 que coloca en la base, y una barra adicional de 1×1 que coloca en la segunda columna.

Ayuda a Antonio escribiendo un programa que calcule el mínimo **número de barras horizontales** necesarias para formar una configuración dada.

Entrada

La primer línea contiene un entero T, el número de casos. Por cada caso, dos líneas, la primera con dos enteros R y C separados por un espacio. La segunda línea contiene C enteros (H_i, con $1 \leq i \leq C$ y $1 \leq H_i \leq R$) separados por espacios, que representan la altura de cada columna en dicha configuración.

Salida

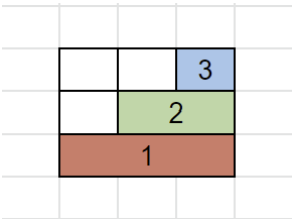
Por cada caso en la entrada, una línea con un entero, el mínimo número de barras necesarias para formar la configuración.

Ejemplo

input	output	description
3 3 3 1 2 3 3 3	3 2 3	Vea las figuras al final para entender la salida.

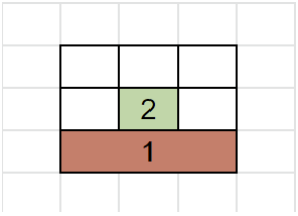
input	output	description
1 2 1 3 3 2 1 2		

- Solución al caso 1



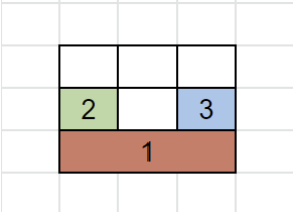
caso 1

- Solución al caso 2



caso 2

- Solución al caso 3



caso 3

Límites

- $1 \leq T \leq 50$

Para 40% de los casos * $1 \leq R, C \leq 1000$

Para 100% de los casos * $1 \leq R, C \leq 10^5$

For o While

Autor: Luis Germán Gutiérrez Torres

Puntos	100	**Límite de memoria**	32 MiB
Límite de tiempo (caso)	1s	**Límite de tiempo (total)**	1m0s
Tamaño límite de entrada (bytes)	10 KiB		

Descripción

Dicen algunos que la única diferencia entre un ciclo for y un ciclo while es la sintaxis para definirlos. Se supone que no hay diferencia de rendimiento en absoluto. Tal vez estás a punto de comprobarlo.

Te piden que realices muchas sumas desde un valor ***A*** hasta un valor ***B***. Por ejemplo, si ***A*** es igual a 250 y ***B*** es igual a 252, tu tarea es sumar 250 + 251 + 252, es decir, tienes que sumar todos los números desde ***A*** hasta ***B*** incluyendo ***A*** y ***B***.

Debes tener cuidado porque no sabes si ***A*** es mayor o menor que ***B***, por ejemplo, ***A*** puede ser 90 y ***B*** puede ser 85 en cuyo caso debes sumar 85+86+87+88+89+90.

Entrada

Recibirás un entero ***N*** que indica la cantidad de sumatorias que debes realizar seguido de ***N*** líneas. En cada línea se te proporcionará el valor de ***A*** y el valor de ***B***.

A y ***B*** son valores enteros.

Salida

Deberás imprimir ***N*** líneas, en cada línea un único valor que representa la sumatoria del rango.

Ejemplo

input	output	description
3	26	5+6+7+8 = 26
5 8	273	
90 92	-27	90+91+92 =273
-10 -8		
		(-10)+(-9)+(-8) = -27

Límites

- $1 <= N <= 30$
- $-10^9 <= A, B <= 10^9$

Suerte

Autor: Ulises Méndez Martínez

Puntos	100	**Límite de memoria**	32 MiB
Límite de tiempo (caso)	1s	**Límite de tiempo (total)**	1m0s
Tamaño límite de entrada (bytes)	50 KiB		

Descripción

Antonio acaba de leer acerca de **números de la suerte**, el leyó que un **número de la suerte** es de la forma $S = 8 \cdots 86 \cdots 6$, donde S tiene al menos un dígito y el número de dígitos 6 y 8 pueden ser cero. Ejemplos de estos números **son**: 8,88,6,66,86,866,8866, etc. Ejemplos de números que **no son** de la suerte: 68,868,3,688.

Antonio se pregunta, dado un entero N, ¿Cuál es el **mínimo** número de la suerte múltiplo de N y cuya longitud no es mayor a L dígitos?.

Ayuda a Antonio creando un programa que le ayude a responder su pregunta.

Entrada

La primer línea contiene un entero T indicando el número de casos. A continuación T líneas, cada una describiendo un caso con dos enteros N y L separados por un espacio.

Salida

T líneas (una por cada caso), por cada línea un entero S, S es la respuesta a la pregunta de Antonio o -1 si dicha solución no existe.

Ejemplo

input	output	description
5	6	
1 10	6	
2 10	6	
3 10	8	
4 10	-1	
5 10		

Límites

- $T \leq 1000$
- $1 \leq N \leq 10000$

Para 30% casos

- $1 \leq L \leq 10$

Para 100% casos

- $1 \leq L \leq 200$

¿Cuántas empresas por mes?

Autor: David Morales Orozco

Puntos	100	**Límite de memoria**	32 MiB
Límite de tiempo (caso)	1s	**Límite de tiempo (total)**	1m0s
Tamaño límite de entrada (bytes)	10 KiB		

Descripción

La clave del registro federal de contribuyentes (RFC) para una persona moral, se compone de la siguiente manera.



Para este ejercicio se requiere tener una lista en la que se indiquen los meses y el número de personas morales que se crearon en ese mes a partir de una lista de RFC proporcionada, para identificar el mes utilice el carácter 6 y 7 de los RFC, proporcionados.

Entrada

Lista de 15 Registros de RFC. Un RFC de 12 caracteres por línea, de acuerdo a la estructura anterior.

Salida

Una lista de meses y el número de empresas que se crearon ese mes de la lista proporcionada. La lista debe iniciar en enero y terminar en diciembre.

Ejemplo

Input	output
MOH820115NI4	ENERO 4
TGJ850121AI3	FEBRERO 1
GAK921218LL2	MARZO 1
AAA920127BB1	ABRIL 0
BGS920206CC5	MAYO 4
CJD820305DE6	JUNIO 4
SLE720111AB5	JULIO 0
ABR720525CA4	AGOSTO 0
ANZ930622CC4	SEPTIEMBRE 0
DML720525TT2	OCTUBRE 0
JEO930622TH3	NOVIEMBRE 0
IXP720525KI9	DICIEMBRE 1
OOQ930622XD8	
LAA720525SA7	
MON930622AA2	

Atsa y las gasolineras

Autor: Orlando Isay Mendoza García

Puntos	100	**Límite de memoria**	32 MiB
Límite de tiempo (caso)	750ms	**Límite de tiempo (total)**	7.5s
Tamaño límite de entrada (bytes)	781.25 KiB		

Descripción

Durante los últimos años, Atsa invirtió sus ganancias de Bitcoin en construir algunas gasolineras al lado de las autopistas. Atsa fue tan exitoso que ahora su empresa tiene gasolineras en muchos estados. Pero Atsa ha decidido que es tiempo de retirarse y dejar el negocio en manos de sus hijos.

Dado que sus hijos tienen ideas muy distintas del rumbo que debe tomar la compañía, Atsa piensa que es mejor repartir las gasolineras entre ellos, para que cada uno pueda administrar de la forma que prefieran.

Para asegurar una repartición justa, Atsa quiere saber si es posible repartir todas las gasolineras entre sus dos hijos, de tal forma que ambos terminen con un grupo de gasolineras que cubra todas las autopistas. Un tramo r se considera cubierto por un conjunto de gasolineras S, si existe al menos una estación s en S que cubra a r. A una estación cubre todos los tramos adyacentes.

Si existe más de una manera de repartir las gasolineras, Atsa quiere encontrar alguna manera que minimize la diferencia entre el número de gasolineras repartidas a cada hijo.

Atsa se encuentra ocupado pensando qué va a hacer después de retirarse, así que te pidió que le ayudas con esta tarea.

Una última pieza de información que Atsa cree que puede ser de ayuda, es que no hay tramos que conecten gasolineras de distintos estados, y que siempre es posible trasladarse entre cualquier par de gasolineras de un mismo estado utilizando los tramos existentes.

Dada la descripción de las autopistas y de las gasolineras, encuentra si es posible separar todas las gasolineras en dos grupos, considerando las restricciones anteriores. En caso de que puedan separar, encuentra alguna repartición que minimize la diferencia.

Entrada

La primera línea contiene dos enteros G y R, indicando el número de gasolineras y el número de tramos respectivamente.

Las siguientes R líneas contienen dos enteros a_i y b_i, que indican que existe un tramo de carretera que une las gasolineras a_i y b_i. Las G gasolineras están numeradas de 0 a $G - 1$.

Salida

Si no es posible repartir todas las gasolineras imprime "NO".

De lo contrario, imprime "YES" en la primera línea. En la segunda línea imprime la mínima diferencia posible. En la tercera y cuarta línea, imprime la cantidad de gasolineras en cada grupo, seguidas de los identificadores que pertenecen al grupo.

Puedes imprimir, cualquier repartición válida.

Ejemplo

input	output
3 2 0 1 0 2	YES 1 1 0 2 1 2

input	output
4 6 0 1 1 3 0 2 2 3 1 2 0 3	NO

Límites

- $1 \leq G \leq 10^5$
- $R \leq 2 * 10^5$
- La cantidad de estados nunca será mayor a 10.

Ganadores del Coding Cup TecNM

Autor: Patricia Vega Flores

Puntos	100	**Límite de memoria**	32 MiB
Límite de tiempo (caso)	1s	**Límite de tiempo (total)**	1m0s
Tamaño límite de entrada (bytes)	10 KiB		

Descripción

ITSUR-i-2 ha sido encomendado a una misión secreta muy importante, será el encargado de comunicar los ganadores del concurso Coding Cup TecNM y le han pedido que por ningún motivo su forma de transmitir el mensaje ocasione que haya confusiones a la hora de informar quiénes fueron los ganadores, por ello, se le ha ocurrido que difundirá los nombres de los equipos los ganadores usando un esquema parecido al lenguaje de desambiguación radiotelefónica que utilizan los militares, el cual fue establecido por la Organización de Aviación Civil Internacional (OACI, ICAO en inglés), agencia de la ONU creada en 1944. También es conocido como Interco y como alfabeto fonético OACI, al que luego llamaron «Alfabeto Internacional de Deletreo Radiotelefónico» (International Radiotelephony Spelling Alphabet, en inglés). Finalmente su forma más común de referirse a él es NATO phonetic alphabet.

Este alfabeto funciona asignando una palabra a cada letra, por ejemplo: N34A se definiría como «November Three Four Alfa».

La tabla de traducción del alfabeto NATO es la siguiente:

Symbol	Code Word	Symbol	Code Word	Symbol	Code Word
A	Alfa	N	November	1	One
B	Bravo	O	Oscar	2	Two
C	Charlie	P	Papa	3	Three
D	Delta	Q	Quebec	4	Four
E	Echo	R	Romeo	5	Five
F	Foxtrot	S	Sierra	6	Six
G	Golf	T	Tango	7	Seven
H	Hotel	U	Uniform	8	Eight
I	India	V	Victor	9	Nine
J	Juliett	W	Whiskey	0	Zero
K	Kilo	X	X-Ray	.	Decimal

Symbol	Code Word	Symbol	Code Word	Symbol	Code Word
L	Lima	Y	Yankee		
M	Mike	Z	Zulu		

Adicionalmente, con la finalidad de hacer más breves las transmisiones, cuando una letra o número se repite se incluirá una indicación adicional para mencionar qué letra o número se repite y cuántas veces, con lo que el ejemplo N334AAA quedaría así: « November double Three Four triple Alfa» Otro ejemplo: 1700000123 se trasmitiría como «One Seven quintuple Zero One Two Tree». Esta disminución solo aplica hasta 5 repeticiones, en caso de haber más se deberá colocar los elementos faltantes con un mecanismo de repetición idéntico reiterativo, por ejemplo: A5c7777777a5, se transmitiría como: «Alfa Five Charlie quintuple Seven double Seven Alfa Five». Por lo anterior, se manejará para las repeticiones los siguientes elementos:

Repeticiones	Code Word
2	double
3	triple
4	quadruple
5	quintuple

Por lo anterior, ITSUR-d2 quiere practicar antes de la ceremonia de premiación así que… ¿Podrías ayudarle a que se le facilite la tarea?

ITSUR-d2 necesita que hagas un programa que le ayude a implementar esta funcionalidad y no se equivoque durante la ceremonia de premiación.

Entrada

Una cadena de texto E que representa la cadena que deberá mencionar ITSUR-d2 usando el alfabeto de desambiguación.

Salida

La primera línea deberá contener una cadena E adaptada a las posibilidades del alfabeto NATO, es decir, existe la posibilidad de que esa cadena de texto tenga elementos que no se manejan en el alfabeto de desambiguación, por ello, ITSUR-d2 deberá asegurarse primero de generar la cadena excluyendo aquellos elementos que no sea posible representar con el alfabeto antes mencionado.

En una segunda línea la cadena de texto T que ayudará a ITSUR-d2 a mencionar la cadena E con el alfabeto NATO.

Ejemplo

input	output
	A5c7777777a5

input	output
	Alfa Five Charlie quintuple Seven double Seven Alfa Five

input	output
Best Tec NM 2.0 ++	BestTecNM2.0 Bravo Echo Sierra double Tango Echo Charlie November Mike Two Decimal Zero

Límites

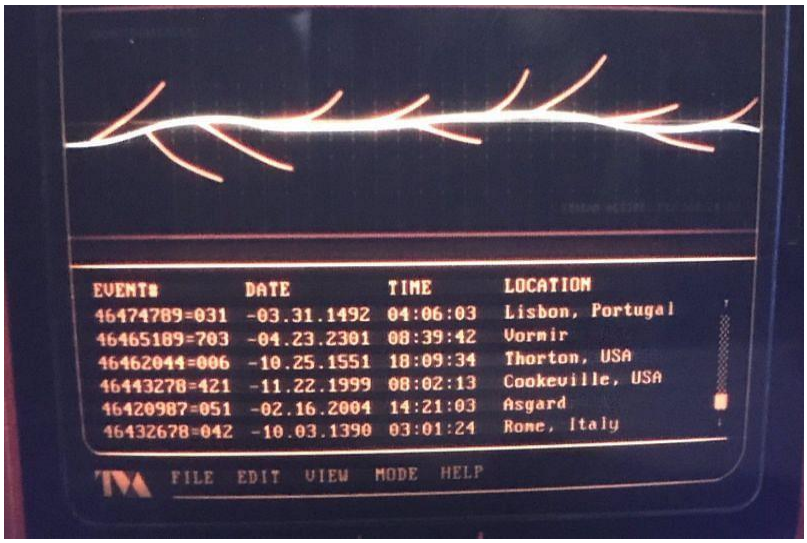
- 1 caracter $\leq E, T \leq$ 1000 caracteres

Linea del Tiempo

Autor: Patricia Vega Flores

Puntos	100	**Límite de memoria**	32 MiB
Límite de tiempo (caso)	1s	**Límite de tiempo (total)**	1m0s
Tamaño límite de entrada (bytes)	195.546875 KiB		

Recientemente conseguiste trabajo como oficinista en una de esas agencias policiales del tiempo ambientadas en el siglo 20. Tu trabajo consiste en darle seguimiento a varias personas y objetos a través del tiempo. Desafortunadamente, los agentes policiales no hacen su trabajo como lo hacían antes y ahora la línea del tiempo empezó a divergir y a tener ramas; al menos las ramas no han tenido contacto entre ellas aún...



Lo que se ve en tu monitor

La agencia es algo quisquilloza y tiene un formato para darle seguimiento a la lista de cosas notables, con un par de columnas, que guardan la lista de cosas a la que se les da seguimiento y una descripción para estas. Los nombres en la tabla siempre están ordenados lexicográficamente por el código ASCII de cada caracter.

Nombre	Descripción
Abigail	Persona comun y corriente que vive en Aberdeen
BART	Sistema de Transito Rapido del Area de la Bahia
Bart	Habitante de Springfield

Y siendo todavía más burocrática, la agencia tiene un método para darle un identificador único a cada estado del mundo (tabla). Esto lo logran concatenando todos los nombres, y a eso concatenándole todas las descripciones. De aquí se toman los bits de toda esa cadena (que está codificada en ASCII con 8 bits por caracter) y se obtiene un número (`w` por ejemplo en bits es `01110111`). Ese número se toma módulo $10^9 + 7$, y ese es el identificador de la tabla. Una tabla vacía está identificada por el número 0.

Por ejemplo, si la tabla fuera:

Nombre	Descripción
x	z
y	w

La cadena concatenada es `xyzw`, la cual al convertirla en bits se obtiene `01111000011110010111101001110111`. Este número es igual a `2021227127` en base 10, y al tomarlo módulo $10^9 + 7$, se obtiene 21227113.

Para tu trabajo se te dará una lista de operaciones para hacer con las tablas (identificadas por el número descrito arriba). Después de cada operación con las tablas tienes que responder con el ID de la nueva tabla generada para ciertas operaciones. Para las peticiones tendrás que responder la información que se te pide.

Entrada y Salida

En la primera línea un número Q, el número de operaciones a realizar.

En las siguientes Q líneas, una de las siguientes tipos de entradas:

- `A ; <id> ; <nombre> ; <descripcion>`: añade un nombre `<nombre>` con descripción `<descripcion>` a la tabla con id `<id>`. Esto crea una tabla nueva en lugar de realmente editarla. Se debe imprimir el id de la nueva tabla generada de acuerdo a las reglas descritas arriba.
- `E ; <id> ; <nombre> ; <descripcion>`: edita la descripción del sujeto con nombre `<nombre>` dejando la descripción `<descripcion>` en la tabla con id `<id>`. Esto crea una tabla nueva en lugar de realmente editarla. Se debe imprimir el id de la nueva tabla generada de acuerdo a las reglas descritas arriba.
- `D ; <id> ; <nombre>`: borra la fila con nombre `<nombre>` con descripción `<descripcion>` a la tabla con id `<id>`. Se debe imprimir el id de la nueva tabla generada de acuerdo a las reglas descritas arriba. Esto crea una tabla nueva en lugar de realmente editarla. Si el nombre no existe en esta tabla, se debe imprimir `NNE` en su lugar.
- `P ; <id> ; <nombre>`: Petición para imprimir la descripción con nombre `<nombre>` de la tabla con id `<id>`. Si el nombre no existe en esta tabla, se debe imprimir `NNE` en su lugar.

Para todas las peticiones, si el id solicitado no existe, se debe imprimir `IDNE` en su lugar.

input	output
7 A ; 0 ; y ; w A ; 31095 ; x ; z E ; 21227113 ; y ; A D ; 21227113 ; y P ; 30842 ; x P ; 30842 ; y A ; 9 ; x ; y	31095 21227113 21227059 30842 z NNE IDNE

input	output
3 A ; 0 ; BART ; Sistema de Transito Rapido del Area de la Bahia A ; 114190815 ; Bart ; Habitante de Springfield A ; 172823486 ; Abigail ; Persona comun y corriente que vive en Aberdeen	114190815 172823486 943654621

Límites

- Para todos los casos, $1 \leq Q \leq 3 \times 10^5$.
- Los ids en las peticiones siempre serán enteros no-negativos menores a $10^9 + 7$.
- Nunca se te pedirá añadir un nombre ya existente a una tabla.
- Los caracteres en la entrada serán convertibles a ASCII con valores entre 12 y 127 distintos a punto y coma.
- El total de caracteres en los nombres y descripciones en la entrada no excederá 10^5.
- Para un 10% de los casos, el total de tablas generadas nunca será mayor a 8.
- Para otro 30% de los casos, el total de tablas generadas nunca será mayor a 1000.
- Para el 60% restante de los casos no habrá restricciones en cuanto al total de tablas generadas.

ITSUR-d2 y su travesía en Chamaeleontis

Autor: Patricia Vega Flores

Puntos	100	**Límite de memoria**	32 MiB
Límite de tiempo (caso)	1s	**Límite de tiempo (total)**	1m0s
Tamaño límite de entrada (bytes)	10 KiB		

Descripción

ITSUR-d2 es un robot explorador muy intrépido que le gusta explorar mundos desconocidos, en sus viajes de exploración es muy arriesgado, pues en ocasiones ha estado a punto de destruirse enfrentándose a situaciones muy adversas.

Durante sus diversos viajes ITSUR-d2 se ha enterado que posiblemente tenga un pariente, es decir, se ha enterado que en el planeta lejano Chamaeleontis habita el robot Karel que fue creado por la misma tecnología de inteligencia artificial que él.

ITSUR-d2 está muy emocionado y ha decidido viajar miles de años luz para conocer a Karel. Al llegar al planeta Chamaeleontis, ITSUR-d2 se da cuenta que este planeta está plagado de especies peligrosas y con mucha destreza y suerte ha podido llegar a lo que parece ser una fortaleza donde habita Karel, en la entrada hay una advertencia que ha logrado descifrar y dice:

"Si a Karel quieres encontrar,

el laberinto has de cruzar,

pero no te has de confiar

si una ruta te ha de recompensar

porque tienes que saber

que en el camino puedes perecer"

ITSUR-d2 está totalmente decidido a superar el laberinto a pesar de las advertencias del acertijo. Su estrategia consiste en enviar a pequeños robots exploradores que le informen sobre la situación y establecer una ruta segura, sobre todo porque el interior es de una negrura intensa que no deba ver nada. Después de sus primeros intentos, los exploradores han dejado de responderle, pero, gracias a ellos, se ha dado cuenta que el laberinto tiene pistas, trampas y recompensas.

Al parecer, las pistas se encuentran en el camino y están iluminadas, por lo que es fácil interpretarlas, los exploradores han encontrado de 2 tipos: G y A, que van

acompañadas de una clave numérica, las de tipo *G* se refieren a la cantidad de giros que debe dar para orientarse en la dirección correcta y seguir desplazándose, además sabe que después de encontrar una pista *G*, ITSUR-d2 debe avanzar un paso en la orientación final y encontrará una pista *A*. Por otro lado, las pistas *A* le indican cuantos pasos debe dar en la dirección donde se encuentra orientado.

ITSUR-d2 se dio cuenta de que a pesar de que los exploradores seguían las pistas, algunos perecían en el camino, es decir, su energía disminuía súbitamente. Aunque también notó que algunos se cargaban súbitamente.

Después de algunas mejoras en su exploración se dio cuenta que al desplazarse justo después de recibir una descarga o una recarga de energía, en el lugar donde la recibía había una marca especial *D* y *R* respectivamente. Estas marcas iban acompañadas de otra clave numérica, en este caso con valores decimales que aparentemente indican el % de recarga o descarga que se recibía.

Acompaña a ITSUR-d2 en su travesía por el laberinto de ***NxM***, ayudándolo a tomar cada una de las pistas *G* y *A*, además de estar al tanto siempre de la cantidad de energía *E*, que será afectada por las situaciones *R* y *D*, hasta llegar a la compuerta con la marca de Karel *K*.

Toma en cuenta que ITSUR-d2 perecerá en el camino si se queda con 0% de energía y en este caso, no podrá llegar a Karel. Como ITSUR-d2 sabe que se enfrenta a una misión muy incierta y desgastante, se asegura de tener su energía al 100% antes de entrar al laberinto, ya que de acuerdo a sus cálculos se ha dado cuenta que avanzar 4 pasos le quitarán 1% de energía y dar 4 giros le consumen 2% de energía.

Considerando lo anterior, ITSUR-d2 quiere ahorrar energía a como dé lugar, de modo que debe dar la menor cantidad de giros que le sea posible.

Por ejemplo: una pista *G*6, implicaría que ITSUR-d2 perdería 3% de energía si da los giros tal cual los piden, pero si quiere ahorrar energía, podría simplemente dar 2 giros quedando en la misma dirección que si diera 6, de este modo solo gasta 1% de energía.

Entrada

Una línea con los enteros ***N*** y ***M*** que representan la dimensión del laberinto, seguida de ***N*** líneas con ***M*** tokens ***T*** separados por espacios.

Los tokens ***T*** pueden ser pistas de la forma: “Ap” o “Gv”, trampas de la forma “Dp”, recompensas de la forma “Rp”, tokens “X” que no sabemos si son abismos o caminos transitables, o un token “K” que representa la compuerta que llevará a ITSUR-d2 directamente con Karel.

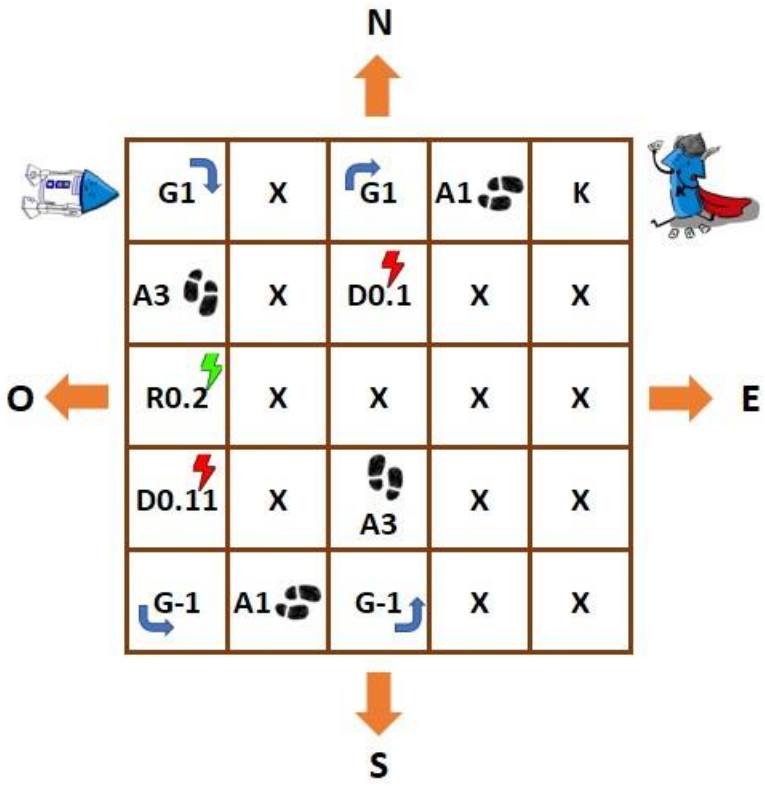
La pista *A* va acompañada de una clave numérica entera ***p***, que indica la cantidad de pasos que debe avanzar o retroceder ITSUR-d2, la pista *G* va acompañada de una clave numérica entera ***v***, que indica la cantidad de giros a la derecha (si es positiva) o la la

izquierda (si es negativa) que dará ITSUR-d2 para orientarse correctamente de acuerdo a la pista.

Las trampas o recompensas van acompañadas de un número decimal ***p*** que indica el porcentaje de recarga o descarga que tendrá que asumir ITSUR-d2 cuando las atraviese, ese porcentaje será aplicado sobre la energía actual, por ejemplo: si cuenta con una energía al 50% y se aplica una recarga R0.5, la energía final será de 100%, también se considera que ITSUR-d2 nunca podrá acumular más del 100% de energía pues tendría el riesgo de sobrecalentarse y perecer. Por ello, aunque tome la recarga ITSUR-d2 deberá desperdiciar el resto de energía que no pueda consumir.

La penúltima línea contendrá 2 enteros ***X*** y ***Y*** que representarán la ubicación de la entrada del laberinto, donde habrá siempre una pista. comenzando desde la esquina superior izquierda.

Finalmente, la última línea indicará la orientación inicial ***O*** de ITSUR-d2 al entrar al laberinto, siendo considerado que las primeras líneas están hacia el Norte y las últimas hacia el Sur. ***O*** puede ser cualquiera de los valores N, S, E, O, correspondientes a Norte, Sur, Este y Oeste, respectivamente.



Laberinto de Chamaeleontis

NOTA: Se asegura que nunca una pista ocasionará que ITSUR-d2 choque con las fronteras del laberinto.

Salida

La primera línea deberá contener la cantidad de pasos que tuvo que caminar ITSUR-d2 hasta llegar a la compuerta de Karel o hasta perecer.

En una segunda línea mostrará alguna de las siguientes variaciones:

- La cantidad de energía ***E*** (en formato entero sin decimales) de ITSUR-d2, separada por un espacio y seguida del mensaje ¡ITSUR-d2 ha conocido a Karel :)! En caso de que haya llegado a Karel con energía.
- El mensaje "ITSUR-d2 ha perecido en el intento de conocer a Karel! :'(" En caso de ITSUR-d2 se haya quedado sin energia antes de poder continuar

Ejemplo

input	output
5 5 G1 X G1 A1 K A3 X X X X X X X X X X X A3 X X G-1 A1 G-1 X X 0 0 E	13 94 ¡ITSUR-d2 ha conocido a Karel :)!

input	output
5 5 G1 X G1 A1 K A3 X D0.5 X X R0.2 X X X X D0.5 X A3 X X G-1 A1 G-1 X X 0 0 E	10 ITSUR-d2 ha perecido en el intento de conocer a Karel! :'(

Límites

- $1 \leq N, M \leq 1000$
- $0 \leq X, Y < 1000$

- $-900 \leq \boldsymbol{p} \leq 900$
- $-100 \leq \boldsymbol{v} \leq 100$
- $0 < \boldsymbol{p} \leq 1$

Superposición

Autor: Patricia Vega Flores

Puntos	100	**Límite de memoria**	32 MiB
Límite de tiempo (caso)	1s	**Límite de tiempo (total)**	1m0s
Tamaño límite de entrada (bytes)	10 KiB		

Descripción

Durante la pandemia Juanito se sentía muy aburrido, pues no podía jugar con nadie personalmente, todo era en línea, así que con sus amigos comenzó a idear un nuevo juego, este consiste en que uno de ellos lanza sobre una mesa una tarjeta rectangular de ***N1xM1***, otro de sus amigos lanza otra tarjeta ***N2xM2***.

El juego requiere mucha destreza pues su objetivo es, primeramente, que ambas tarjetas queden una sobre la otra. Después, deben lograr que ambas tarjetas queden en la misma orientación, es decir, que los lados más anchos de ambas tarjetas estén en el mismo eje.

De esta manera se espera que para poder ganar deben quedar ambas tarjetas sobrepuestas no importa que una esté más grande que la otra, pero la más grande debe contener a la más pequeña.

Afortunadamente, Juanito y sus amigos han adquirido la gran habilidad de que cada tarjeta lanzada siempre quedará alineada a la mesa.

Ahora han pensado que se divertirán aún más haciendo un programa que los ayude a detectar si ganan o pierden.

A continuación se muestran algunos ejemplos de superposición:



Ejemplos

Se han planteado que van a manejar la mesa como un plano cartesiano e identificarán las tarjetas con las coordenadas de los 4 vértices.

Entrada

Una línea 8 números enteros ***T1X1***, ***T1Y1***, ***T1X2***, ***T1Y2***, ***T1X3***, ***T1Y3***, ***T1X4***, ***T1Y4***, que representan los 4 pares de coordenadas X y Y que representan los vértices de la primera tarjeta.

En la segunda línea otros 8 números enteros ***T2X1***, ***T2Y1***, ***T2X2***, ***T2Y2***, ***T2X3***, ***T2Y3***, ***T2X4***, ***T2Y4***, que representan los 4 pares de coordenadas X y Y que representan los vértices de la segunda tarjeta.

Salida

Cuando ambas tarjetas son de las mismas dimensiones y se encuentran en las mismas posiciones se imprimirá en una sola línea el mensaje "HAN GANADO Ambas tarjetas han coincidido".

En caso de que una tarjeta sea de menor tamaño que la otra, pero estén totalmente superpuestas se colocará uno de los siguientes mensajes que corresponda "HAN GANADO La tarjeta 1 esta contenida en la tarjeta 2" o "HAN GANADO La tarjeta 2 esta contenida en la tarjeta 1".

Por último, cuando las tarjetas no se superpongan completamente imprimirá "HAN PERDIDO".

Ejemplo

input	output
4 1 1 1 1 2 4 2	HAN GANADO La tarjeta 1 esta contenida en la tarjeta 2

input	output
5 3 0 0 5 0 0 3	

input	output
0 2 3 1 3 2 0 1	HAN PERDIDO
1 3 2 3 2 0 1 0	

Límites

- $0 \leq \boldsymbol{X}, \boldsymbol{Y} < 1000$